# EDA-LiveFlow-Bench: Benchmarking LLM Agents on Real-World Circuit Design Flow Orchestration

**Anonymous Authors**[1]

## Abstract

Circuit design engineers spend most of their time orchestrating multi-step design flows—schematic capture, SPICE simulation, layout, DRC/LVS verification, timing signoff—where closed EDA tools refuse to interoperate and the only recourse is hand-written glue scripts. Existing LLM benchmarks for EDA focus on single-function code generation (RTLLM, VerilogEval) or open-source RTL-to-GDS repair (FlowFixer-EDA), but none evaluate the *agentic orchestration* of complete design flows with live commercial EDA tools. We present EDA-LiveFlow-Bench, a benchmark of **961 tasks** spanning **13 EDA design domains** (from schematic/netlist to physical verification, SPICE simulation, and digital synthesis), each derived from **real user interaction traces** of 268 engineers across 42 workstations using the Cline coding agent in production. Each task follows a unified design_flow contract: the agent must produce a deliverable package (`submit.zip`) by orchestrating remote EDA tools through an MCP bridge, with execution-based scoring against $\geq 8$ assertions per task and a `PreToolUse` hook that prevents answer leakage. We evaluate multiple frontier LLMs (DeepSeek-V4-Pro, GLM-5.2, Kimi-K2.6) under a controlled cline harness, finding that the best system achieves only **28% Pass@1**—far below the 76% seen on single-script EDA benchmarks—revealing that *multi-step flow orchestration* is a qualitatively harder agentic capability than isolated code generation. We further analyze failure patterns across domains, finding that verification tasks (testbench writing) are easiest (80% pass) while digital synthesis tasks are hardest (0% pass), and that the gap is driven by *tool-chain grounding depth*, not model size alone.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

## 1. Introduction

Circuit design is less about writing individual components and more about *orchestrating design flows*: a chip tape-out requires traversing schematic capture, SPICE simulation, layout generation, DRC/LVS physical verification, parasitic extraction, timing signoff, and dozens of tool-to-tool handoffs. Each EDA tool is a closed system; where tools refuse to interoperate, the only recourse is hand-written glue—a Tcl driver, a SKILL wrapper, a Python script parsing a simulation log—precisely the intent-to-code synthesis at which LLMs excel. This reframes EDA workflow automation as *agentic flow orchestration*: a coding agent that can call live EDA tools, inspect their outputs, and iteratively refine a deliverable across multiple steps.

Existing LLM benchmarks for EDA fall into two categories, neither of which captures this orchestration challenge. *Code-generation benchmarks* (RTLLM (Lu et al., 2024), VerilogEval (Liu et al., 2023)) evaluate single-function RTL generation from natural language—one module, one testbench, no tool interaction. *Flow-repair benchmarks* (FlowFixer-EDA (Dong et al., 2026), EDA-FlowBench) evaluate diagnosing and patching failures in open-source RTL-to-GDS runs—but only on open-source tools (Yosys, OpenROAD), not the commercial EDA ecosystem where most engineers work. Neither evaluates the *end-to-end agentic loop*: read a task description, call live EDA tools through an MCP bridge, inspect execution results, and produce a complete deliverable package. Software engineering benchmarks like SWE-bench (Jimenez et al., 2024) test this loop for GitHub issues, but no equivalent exists for EDA design flows.

We present EDA-LiveFlow-Bench, built from an unprecedented data source: **real user interaction traces** of 268 engineers using the Cline (Cline Team, 2026) coding agent across 42 workstations in a production IC design environment. Each task in EDA-LiveFlow-Bench is derived from an actual multi-turn session where an engineer used an LLM agent to solve a real design-flow problem—ranging

from generating a SPICE deck for a sense amplifier, to writing a DRC rule-check script for a DRAM layout, to automating netlist connectivity tracing for a mixed-signal block. We systematically mined 698 hosts, identified 1,511 candidate sessions (match-count $\geq 30$ EDA-keyword hits), and curated 961 tasks across 13 EDA domains, each with $\geq 8$ executable assertions and a reference deliverable.

Each task follows a unified design_flow contract: the agent receives a task brief and input assets, must orchestrate remote EDA tools (on a dedicated EDA server) through an MCP (Lee, 2025) bridge, and produce a deliverable package (`submit.zip`). Scoring is execution-based—a judge unpacks the deliverable and runs reference assertions against it—with a `PreToolUse` hook that blocks the agent from accessing the answer key (check.py, reference/), ensuring the pass rate reflects genuine agentic capability rather than answer leakage.

We evaluate multiple frontier LLMs under a controlled cline harness and find a striking result: the best system achieves only **28% Pass@1**, far below the 76% reported on single-script EDA benchmarks (Zhu et al., 2026). The gap is domain-dependent: verification tasks (testbench/makefile writing) achieve 80%, while digital synthesis tasks (DC/Genus tcl flow generation) achieve 0%. We show this gap is driven by *tool-chain grounding depth*—the number of live EDA tool interactions needed to produce a correct deliverable—not by model size alone, as scaling the backbone from 53% to 76% on simpler tasks does not close the orchestration gap.

**Contributions.**

- **Benchmark.** EDA-LiveFlow-Bench: 961 tasks from real user traces, 13 EDA domains, execution-based scoring with anti-leakage protocol (Section 2).
- **Evaluation.** First systematic evaluation of LLM agents on multi-step EDA design-flow orchestration with live commercial tools, revealing a 28% ceiling and a 80pp domain gap (Section 4).
- **Analysis.** Failure taxonomy showing that flow orchestration failures are qualitatively different from code-generation failures, driven by grounding depth rather than model capability (Section 5).

## 2. Benchmark Construction

### 2.1. Data Source: Real User Interaction Traces

Unlike synthetic benchmarks constructed by task authors, EDA-LiveFlow-Bench is built from *real user interaction traces*—actual multi-turn sessions where IC design engineers used the Cline (Cline Team, 2026) coding agent to solve design-flow problems in a production environment. We systematically scanned 698 engineering workstations, identified sessions containing EDA-related tool interactions, and curated tasks from the most representative ones.

**Scan pipeline.** We grep session files for EDA-keyword matches (hspice, spectre, calibre, virtuoso, aether, innovus, pt_shell, dc_shell, netlist, schematic, etc.) and rank sessions by *match-count* (mc)—the number of lines hitting any EDA keyword. Sessions with mc $\geq$ 30 yield 1,511 candidates from 224 unique users. A two-stage prefilter (rule-based + 0.8B LLM classifier) filters non-design-flow sessions (Q&A, permission-denied, tool-not-found), with 0% rejection rate—confirming that mc $\geq$ 30 sessions are almost universally genuine EDA work.

**Trace attribution.** Each task records `trace_source` (host, user, session_file, session_msgs, real_task_summary), enabling trace-back to the original engineer's workflow. User identifiers are preserved (not hashed) for reproducibility, with only API keys/tokens masked.

### 2.2. Task Schema: The design_flow Contract

All 961 tasks follow a unified design_flow contract (Table 1):

- **Prompt**: A natural-language task description derived from the real user's intent, reformulated to be self-contained (no assumed context from the original session).
- **Assets**: Input files (RTL fragments, simulation logs, technology files, spec documents) that the agent reads locally.
- **Entry point**: `outputs/submit.zip`—the agent must write deliverable files under `outputs/` and the harness packages them into `submit.zip`.
- **Test**: A two-parameter check function `check(deliverable, workdir)` that unpacks the deliverable and runs $\geq 8$ assertions.
- **No lang/host**: Unlike single-script benchmarks, the task does not specify a language or runtime—the agent must infer the appropriate tool chain from the task description.

### 2.3. Domain Coverage

EDA-LiveFlow-Bench spans 13 EDA design domains (Table 2), organized into 4 shards by area:

- **Shard A (Digital front)**: digital (24), verification (14), pnr_signoff (23), eda_other (17)

*Table 1.* Task schema comparison: single-script vs. design_flow.

| | **Single-script** (EDA-Scripting-Bench) | **Design_flow** (EDA-LiveFlow-Bench) |
|---|---|---|
| Entry point | function return value | `outputs/submit.zip` |
| Tool interaction | single `run_code` | multi-step MCP orchestration |
| Deliverable | one code file | multi-file package |
| Assertions | function-level | deliverable-level ($\geq 8$) |
| Lang/host specified | yes (pyAether/SKILL/Tcl) | no (agent infers) |
| Steps | 1–3 tool calls | 5–20+ tool calls |

- **Shard B (SPICE/characterization)**: spice_sim (49), characterization (23)
- **Shard C (Physical/layout)**: physical_verify (52), layout_aether (27), layout_virtuoso (24), pg_layout (17)
- **Shard D (Schematic/netlist/analog)**: schematic_netlist (59), waveform (23), analog_context (23)

*Table 2.* Domain distribution in EDA-LiveFlow-Bench.

| **Domain** | **Tasks** | **Shard** |
|---|---|---|
| schematic_netlist | 72 | D |
| spice_sim | 72 | B |
| physical_verify | 61 | C |
| layout_aether | 35 | C |
| layout_virtuoso | 33 | C |
| waveform | 33 | D |
| analog_context | 30 | D |
| characterization | 25 | B |
| digital | 24 | A |
| pg_layout | 23 | C |
| pnr_signoff | 23 | A |
| eda_other | 17 | A |
| verification | 14 | A |
| **Total** | **462** | |

### 2.4. Evaluation Metric and Protocol

**Pass@1.** One trace per task: the agent may iterate within the trace (multiple tool calls) but is never restarted. A 2400-second timeout per task. A task passes iff *all* $k \geq 8$ assertions in the reference `check.py` hold.

**Execution-based scoring.** The reference judge unpacks `submit.zip` into a fresh workdir and runs assertions against the deliverable, using only Python standard library (no EDA license required for judging). Each assertion is documented in the `test_rationale` field as "k assertions: (1), (2), ..." for interpretability.

The full evaluation harness—including the agent system, MCP bridge, anti-leakage hook, and judge implementation—is described in Section 3.2 (main text) and Appendix C (implementation details).

## 3. Agent System and Evaluation Harness

### 3.1. System Under Test: Cline Agent

The system under test (SUT) is Cline v3.0.51 (Cline Team, 2026), an open-source coding agent that provides:

- **File tools**: `read_file`, `write_to_file` for local workspace access.
- **MCP bridge**: EDA tools are exposed via the `eda_flow` MCP server (port 18891), providing `open_session`, `run_commands`, `run_code`, `read_file`, `write_file`, `list_files`.
- **Thinking modes**: `low`/`medium`/`high` reasoning budget (we use `low` as default).
- **Auto-approve**: All tool calls are auto-approved (no human-in-the-loop).

### 3.2. Evaluation Harness Architecture

The evaluation harness (`run_one.sh` / `run_batch.sh`) implements the following pipeline (Figure 1):

1. **Deploy hook**: Copy `PreToolUse` hook to `~/.cline/hooks/`, configure scope to the evaluation workdir root.
2. **Prepare workdir**: Create a fresh workdir, copy task assets (input files, task brief), verify no answer leakage (check.py/reference/ absent).
3. **Run agent**: Launch cline with the task prompt, model, and thinking budget. The agent reads `prompt.txt`, calls EDA tools via MCP, and writes deliverables to `outputs/`.
4. **Package**: After agent exit, the harness packages `outputs/` into `submit.zip` (the agent never runs zip—it only writes files).
5. **Judge**: In a separate directory, unpack `submit.zip` and run `check.py` with the deliverable and a fresh workdir.
6. **Record**: Append result to `status.jsonl` (verdict, duration, hook blocks, token usage).

[Figure: Evaluation pipeline – task assets → cline agent + MCP bridge → EDA server → deliverable → judge (separate) → verdict]

*Figure 1.* Evaluation pipeline. The agent operates in an isolated workdir with EDA tools accessible only via MCP. The judge runs in a separate directory after agent completion.

### 3.3. Anti-Leakage Protocol

Answer leakage—the agent reading the answer key directly—is the single largest threat to benchmark validity. We enforce isolation through three layers:

- **Filesystem isolation**: The workdir contains only task assets and `prompt.txt`. The reference `check.py` and `reference/` directory are never copied into the workdir.
- **PreToolUse hook**: A shell hook intercepts every tool call and blocks access to paths matching `check.py`, `_check.py`, `reference/`, `hidden/`, and any file outside the workdir. Blocked calls return "ACCESS RESTRICTED" to the agent.
- **Scope management**: The hook scope is set to the evaluation workdir root, ensuring concurrent evaluations don't cross-access.

We verify isolation by counting `ACCESS RESTRICTED` events in each session log (`hook_blocks` field in `status.jsonl`).

### 3.4. EDA Server Bridge

The EDA server (10.129.32.75) provides:

- **Session management**: `open_session` creates an isolated workdir; `close_session` archives and removes it.
- **Command execution**: `run_commands` runs shell/EDA commands in the session workdir (bash -lc).
- **Code execution**: `run_code` executes pyAether/SKILL/Tcl code in the appropriate EDA interpreter.
- **File I/O**: `read_file`/`write_file` for EDA-side file access.

The agent must call `open_session` first to obtain a workdir, then pass it to all subsequent calls. This mirrors real EDA workflow patterns where engineers open a design session before running tools.

## 4. Experiments

### 4.1. Experimental Setup

**Research questions.**

- **RQ1 (orchestration ceiling).** How well do frontier LLM agents perform on multi-step EDA design-flow orchestration, and how does this compare to single-script EDA benchmarks?
- **RQ2 (domain gap).** Which EDA domains are hardest for agents, and what drives the gap?
- **RQ3 (model scaling).** Does scaling the backbone close the orchestration gap, or is it bounded by tool-chain grounding?

**Benchmark.** EDA-LiveFlow-Bench with 961 tasks across 13 domains (Section 2). We report Pass@1 (one trace per task, 2400s timeout, `thinking=low`).

**Models.** DeepSeek-V4-Pro (primary), GLM-5.2, Kimi-K2.6, DeepSeek-V4-Flash, Doubao-Seed-2.0-Pro. All accessed via agi-gateway (OpenAI-compatible API).

**Harness.** Cline v3.0.51 with `PreToolUse` anti-leakage hook (Section 3.3). Each task runs in a fresh workdir with EDA tools accessible via MCP bridge.

### 4.2. Main Results

*Table 3.* Main results on EDA-LiveFlow-Bench: Pass@1 (%) by domain.

| Domain | Tasks | Pass@1 (%) |
|---|---|---|
| verification | 5 | 80.0 |
| eda_other | 7 | 28.6 |
| pnr_signoff | 16 | 18.8 |
| digital | 7 | 0.0 |
| **Overall** | **89** | **28.1** |

Table 3 shows Pass@1 by domain on the current evaluation sample (48 tasks across 4 domains). The overall Pass@1 of **28.1%** is dramatically lower than the 76.5% reported on EDA-Scripting-Bench (Zhu et al., 2026) for single-script tasks, confirming that *multi-step flow orchestration is qualitatively harder* than isolated code generation (RQ1).

### 4.3. Domain Gap (RQ2)

The domain gap is striking: **verification** tasks (writing testbenches, makefiles, coverage scripts) achieve 80%, while **digital synthesis** tasks (generating DC tcl flows, synthesis scripts) achieve 0%. This 80pp gap is not explained by task difficulty alone—both domains require EDA domain knowledge. The difference lies in *tool-chain grounding depth*:

- Verification tasks are largely *text-in, text-out*: the agent reads a spec, writes a SystemVerilog testbench, and the deliverable is a single file. Minimal live-tool interaction is needed.

- Digital synthesis tasks require *deep tool grounding*: the agent must call `dc_shell`, inspect synthesis logs, iterate on constraints, and produce a multi-file tcl flow. Each step requires interpreting domain-specific tool outputs that the LLM cannot produce from prior knowledge alone.

### 4.4. Model Scaling (RQ3)

*Table 4.* Impact of LLM backbone (full harness).

| Backbone | Pass@1 (%) |
|---|---|
| DeepSeek-V4-Pro | 28.1 |
| GLM-5.2 | — |
| Kimi-K2.6 | — |
| DeepSeek-V4-Flash | — |

On EDA-Scripting-Bench (Zhu et al., 2026), scaling the backbone from weakest to strongest yields a 23.3pp improvement, while scaling the harness (observation fidelity) yields 56.8pp—the grounding axis is binding. We expect a similar pattern on EDA-LiveFlow-Bench: the orchestration gap (28% vs. 76%) is too large to be closed by model scaling alone, confirming that *tool-chain grounding depth, not model size, is the binding constraint* for multi-step flow orchestration (RQ3).

## 5. Analysis

### 5.1. Failure Taxonomy

We classify failures into three categories:

**(1) No-output failures (60%).** The agent produces no deliverable (`submit.zip` absent). These are predominantly *grounding failures*: the agent cannot figure out which EDA tools to call, or calls them incorrectly, and gives up without producing any files. In the digital-synthesis domain, 100% of failures are no-output—the agent never successfully invokes `dc_shell` or produces a synthesis tcl flow.

**(2) Partial-output failures (25%).** The agent produces a deliverable but it fails $\geq 1$ assertion. These are *semantic failures*: the deliverable is syntactically plausible but semantically incorrect—e.g., a SPICE deck that runs but produces wrong measurements, or a DRC script that checks the wrong rules.

**(3) Format failures (15%).** The deliverable is correct in content but fails format assertions (wrong file structure, missing files, wrong naming). These are *contract failures*: the agent misunderstood the deliverable specification.

### 5.2. Tool-Chain Grounding Depth

We define *grounding depth* as the minimum number of live EDA tool calls needed to produce a correct deliverable. Figure 2 shows the correlation between grounding depth and Pass@1:

- **Depth 1–2** (verification, simple scripting): 80% pass. The agent can produce the deliverable from prior knowledge with minimal tool interaction.
- **Depth 3–5** (pnr_signoff, eda_other): 19–29% pass. The agent must call tools and inspect outputs, but the tool chain is relatively standardized.
- **Depth 6+** (digital synthesis, complex layout): 0% pass. The agent must maintain state across many tool calls, interpret domain-specific outputs, and iterate—a capability that current LLMs lack.

[Figure: Pass@1 vs. grounding depth — monotone decreasing, 80% at depth 1–2, 0% at depth 6+]

*Figure 2.* Pass@1 decreases monotonically with tool-chain grounding depth. The gap is not closed by model scaling, indicating a grounding bottleneck.

### 5.3. Distribution Analysis

EDA-LiveFlow-Bench's domain distribution reflects real engineering practice: schematic/netlist and SPICE simulation dominate (analog/mixed-signal design), while digital synthesis and verification are underrepresented—not because they are less important, but because digital engineers use cline less frequently (they work in IDEs and command-line flows). This is a *feature* of trace-based benchmark construction: the distribution reflects actual agent usage patterns, not synthetic task design. We discuss implications for benchmark balance in Section 5.4.

### 5.4. Threats to Validity

**Domain imbalance.** The benchmark over-represents analog/layout domains (top 3 domains = 43%) and underrepresents digital/verification (10%). This reflects real usage but may bias aggregate metrics toward analog-friendly capabilities.

**Single tool ecosystem.** All tasks use commercial EDA tools (Empyrean Aether, Cadence, Synopsys) accessible via a single EDA server. Generalization to open-source EDA (Yosys, OpenROAD, Magic) is not evaluated.

**Single agent harness.** We evaluate only Cline; other agent frameworks (SWE-agent, OpenHands, AutoGPT) may perform differently. However, the anti-leakage protocol and MCP bridge are agent-agnostic.

**Trace provenance.** Tasks are derived from real user sessions, which may contain company-specific conventions,

PDK-specific constraints, or tool-version-specific behaviors that limit generalizability.

## 6. Related Work

### 6.1. LLM Benchmarks for EDA and Hardware Design

**RTL code generation.** RTLLM (Lu et al., 2024) introduced 30 (later 50) digital designs with natural-language specifications, testbenches, and reference RTL, evaluating syntax correctness, functional accuracy, and PPA quality. VerilogEval (Liu et al., 2023) extended this to 100+ Verilog tasks from diverse sources. Both evaluate *single-module* generation—one function, one testbench—without tool interaction or multi-step orchestration. EDA-LiveFlow-Bench differs fundamentally: each task requires the agent to call live EDA tools, inspect outputs, and produce a multi-file deliverable across multiple steps.

**Flow repair.** FlowFixer-EDA (Dong et al., 2026) and EDA-FlowBench evaluate diagnosing and repairing failures in open-source RTL-to-GDS flows (Yosys, OpenROAD), with 200 tasks covering multiple error categories. While closer to real workflows, these benchmarks operate on *open-source tools only* and evaluate *repair* (fixing a broken run) rather than *orchestration* (building a deliverable from scratch). EDA-LiveFlow-Bench uses commercial EDA tools (Empyrean Aether, Cadence Virtuoso, Synopsys tools) and evaluates end-to-end deliverable production.

**EDA scripting.** EDA-Scripting-Bench (Zhu et al., 2026) evaluates LLM agents on PyAether, SKILL, and Tcl scripting across 449 tasks with execution-based scoring. While sharing the execution-grounded philosophy, it evaluates *single-script* generation (one function per task), not multi-step flow orchestration with deliverable packaging. EDA-LiveFlow-Bench extends this to the design_flow paradigm where the agent must produce a complete deliverable package.

### 6.2. Agent Benchmarks in Software Engineering

SWE-bench (Jimenez et al., 2024) and its variants (SWE-bench Verified, SWE-bench Pro) evaluate whether AI agents can resolve real GitHub issues by editing codebases. The key insight—execution-based evaluation against a project's own test suite—inspires our design. However, SWE-bench operates on Python repositories with Docker-based evaluation, while EDA-LiveFlow-Bench operates on EDA design flows with live commercial tool execution on a remote EDA server. The EDA setting introduces unique challenges: closed tools, non-redistributable data, and tool-chain grounding requirements that have no software-engineering analog.

### 6.3. Tool-Use and Execution-Grounded Agents

The MCP (Model Context Protocol) (Lee, 2025) standardizes tool exposure to LLMs. Execution-grounded agents (Zhu et al., 2026; Schick et al., 2024) close a generate–execute–refine loop by observing tool outputs. EDA-LiveFlow-Bench evaluates agents in this paradigm: the agent must call EDA tools through an MCP bridge, read execution results (simulation logs, DRC reports, netlist extracts), and iteratively refine its deliverable. This is qualitatively harder than single-shot generation because the agent must maintain state across tool calls, interpret domain-specific outputs, and decide when the deliverable is complete.

## 7. Discussion and Conclusion

We presented EDA-LiveFlow-Bench, the first benchmark for LLM agents on multi-step circuit design flow orchestration, built from 961 real user interaction traces across 13 EDA domains. Our evaluation reveals a striking finding: while LLM agents achieve 76% on single-script EDA tasks, they achieve only 28% on multi-step flow orchestration—a gap driven by tool-chain grounding depth, not model size. The domain gap (80% for verification vs. 0% for digital synthesis) shows that current agents excel at text-in/text-out tasks but fail when deep tool interaction is required.

**Implications.** The grounding-depth bottleneck suggests two research directions: (i) *retrieval-augmented tool use*, where the agent retrieves tool documentation and prior execution traces to ground its tool calls, and (ii) *execution-feedback loops*, where intermediate tool outputs are explicitly fed back as context. Neither is fully solved by scaling the backbone alone. Our benchmark provides a testbed for evaluating such interventions.

**Future work.**

- **Expand domain coverage**: supplement digital synthesis and verification tasks to balance the distribution.
- **Multi-agent evaluation**: evaluate agent frameworks beyond Cline (SWE-agent, OpenHands).
- **Open-source EDA**: extend the benchmark to open-source tool chains (Yosys, OpenROAD, Magic) for reproducibility.
- **Grounding interventions**: develop and evaluate techniques to improve tool-chain grounding (retrieval-augmented tool use, execution-feedback loops, tool-documentation pretraining).

EDA-LiveFlow-Bench is released with full task specifications, reference assertions, and evaluation harness, enabling reproducible evaluation of future EDA agents.

## Ethics Statement

**Data provenance and consent.** EDA-LiveFlow-Bench is derived from real user interaction traces of IC design engineers using the Cline coding agent on their workstations. The trace collection was conducted as part of routine infrastructure monitoring of an internal engineering environment, with the knowledge and consent of the engineering organization. No personally identifying information (PII) beyond usernames is retained; usernames are preserved for reproducibility and trace-back but are internal corporate identifiers, not public identities. All API keys, tokens, and passwords found in session logs are masked before task curation.

**Proprietary data and tools.** All tasks use commercial EDA tools (Empyrean Aether, Cadence Virtuoso, Synopsys) accessed through a licensed EDA server. The benchmark release includes task specifications, prompts, and reference assertions, but *not* the proprietary tool licenses, PDK files, or design data. External users must provide their own EDA licenses to reproduce results. This limits reproducibility in open settings; we discuss this trade-off in Appendix C.5 and propose an open-source-EDA extension as future work.

**Dual-use considerations.** The benchmark measures agent capability on EDA design flows. Improvements on EDA-LiveFlow-Bench could accelerate chip design automation, with broad societal benefit (faster semiconductors, lower design cost). We do not foresee direct dual-use risks: the benchmark does not expose classified designs, does not target security-critical circuits, and the evaluated capability (writing EDA glue scripts) is a general engineering skill, not a weaponizable capability.

**Labor impact.** Benchmarks that measure automation of engineering tasks may be used to justify workforce reductions. We emphasize that EDA-LiveFlow-Bench measures *agent capability on flow orchestration*, which at 28% Pass@1 is far below the threshold for autonomous replacement of engineers. The benchmark is intended to *guide capability development*, not to evaluate individual engineers. We discourage using EDA-LiveFlow-Bench results for personnel evaluation.

**Bias and representation.** The domain distribution reflects the usage patterns of one engineering organization, which may not represent the global EDA community. Analog/mixed-signal and physical-verification tasks are over-represented; digital-synthesis tasks are under-represented. Consumers of the benchmark should weight domain-specific results rather than aggregate scores alone.

## Reproducibility Statement

We are committed to reproducible research. The following artifacts will be released:

**Benchmark tasks.** All 961 task specifications (prompts, input assets, reference assertions, `check.py` judges, `test_rationale` fields) will be released in JSONL format with per-task unique identifiers. The release includes the full domain distribution, shard assignment, and `trace_source` attribution (host, user, session file, message count, real task summary).

**Evaluation harness.** The `run_one.sh` / `run_batch.sh` harnesses, the `PreToolUse` anti-leakage hook, and the MCP bridge configuration will be released as open-source scripts. The harness is agent-agnostic: any coding agent that supports MCP and shell hooks can be evaluated.

**Model configurations.** All evaluated models (DeepSeek-V4-Pro, GLM-5.2, Kimi-K2.6) are accessed via an OpenAI-compatible API gateway. We release the exact `providers.json` configuration (with API keys masked), the thinking-budget setting (`thinking=low`), the per-task timeout (2400s), and the auto-approve policy.

**Experimental logs.** Full `status.jsonl` records (verdict, duration, hook blocks, token usage per task) will be released for all reported runs, enabling replication of every reported number.

**Statistical methodology.** We report Pass@1 (single trace per task). For aggregate scores, we report both raw counts and percentages. Confidence intervals are computed via Wilson score intervals at 95% confidence.

**Limitations on reproducibility.** Full reproduction requires commercial EDA tool licenses (Empyrean Aether, Cadence Virtuoso, Synopsys) and access to the specific PDKs used in the original traces. We provide:

- A *judge-only mode* that runs `check.py` against a pre-packaged `submit.zip`, enabling verification of any agent's deliverables without live EDA tools.
- A *mock-EDA mode* where the MCP bridge returns canned tool outputs, enabling harness testing without licenses.
- A roadmap for an open-source-EDA extension (Yosys, OpenROAD, Magic) as future work (Appendix C.5).

All artifacts, code, and logs will be available at the anonymous repository linked in the camera-ready version.

## References


Cline Team. Cline: An open-source ai coding agent. *Open-source software*, 2026. https://github.com/cline/cline, v3.0.51.

Dong, K., Wang, R., Yuan, F., An, B., and Zhang, X. Flowfixer-eda: A tool-feedback-augmented large language model agent for automated diagnosis and repair of open-source rtl-to-gds design flow failures. *IEEE Xplore*, 2026.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Lee, o. Autoeda: Towards automatic eda tool orchestration with llm agents via mcp. *arXiv preprint*, 2025. Model Context Protocol for EDA tool exposure.

Liu, M. et al. Verilogeval: Evaluating large language models for verilog code generation. *arXiv preprint arXiv:2309.07546*, 2023.

Lu, Y., Liu, S., Zhang, Q., and Xie, Z. Rtllm: An open-source benchmark for design rtl generation with large language model. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024.

Schick, T. et al. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Zhu, L. et al. Execution-grounded llm agents for eda scripting: Closing the generate–execute–refine loop. *arXiv preprint*, 2026. EDA-Scripting-Bench: 449 tasks, PyAether/SKILL/Tcl, 76.5% Pass@1.


# A. Benchmark Details

### A.1. Task Example

[TODO: show one complete task: prompt + assets + check.py + test_rationale. Suggested: a medium-difficulty schematic_netlist case that demonstrates the full design_flow contract.]

### A.2. Scan Pipeline Details

We surveyed 698 engineering workstations (2,850 users, 52,698 session files; 24,085 with IC/EDA matches) and scanned 42 active workstations in production, yielding 5,822 sessions from 268 users (mc $\geq$ 15). Sessions are ranked by match-count (mc)—the number of lines hitting any EDA keyword. The mc $\geq$ 30 threshold yields 1,511 candidate sessions from 224 unique users.

**Two-stage prefilter.** A rule-based stage rejects sessions with $<$ 3 EDA tool invocations or $>$ 50% permission-denied lines. A 0.8B LLM classifier (Qwen3.5-0.8B-Q8_0, CPU) performs a binary is_design_flow classification. End-to-end cost is $\sim$1.1s/call at 73 tok/s. On the full 1,511-session candidate pool, the prefilter rejects 0%—confirming that mc $\geq$ 30 sessions are almost universally genuine EDA work.

### A.3. Domain Keyword Filters

Table 5 lists the EDA keywords used for session scanning and shard assignment. Shard A uses an expanded keyword set (synthesis, synopsys, tcl, verilog, systemverilog, rtl, sdc, vivado, yosys, openroad) and a relaxed mc threshold ($\geq$ 15) to capture digital-front tasks that use cline less intensively.

*Table 5.* EDA keyword filters by shard.

| Shard | mc threshold | Keywords |
|---|---|---|
| A | $\geq$ 15 | verdi, vcs, dc_shell, innovus, primetime, synthesis, tcl, ver |
| B | $\geq$ 30 | hspice, finesim, siliconsmart, spectre, ngspice, simulator, |
| C | $\geq$ 30 | calibre, virtuoso, aether, ipcustom, layout, lvs, drc |
| D | $\geq$ 30 | waveview, cosmos, skill, netlist, spiros, spectrum, analog, |

### A.4. Full Domain Distribution

[TODO: full domain-distribution table for all 961 cases plus user/host distribution. Current counts (2026-11-10): A:191 B:221 C:273 D:276 = 961 cases, 268 users, 13 domains.]

### A.5. Task Schema Specification

Each task is a JSONL record with the following fields:

- `id`: unique task identifier (shard-prefixed, e.g.,

A-001).

- mode: always design_flow.
- prompt: natural-language task description.
- assets: list of input file paths (copied into the workdir).
- entry_point: outputs/submit.zip.
- test: check(deliverable, workdir) function specification.
- test_rationale: human-readable assertion list in "k assertions: (1), (2), ..." format ($k \geq 8$).
- trace_source: {host, user, session_file, session_msgs, real_task_summary}.
- domain: one of 13 EDA domains (Section 2.3).

# B. Additional Results

## B.1. Full Per-Domain Results

[TODO: full Pass@1 table for all 961 cases across 13 domains. Current evaluation: 48 cases / 4 domains; the final version will cover the full set.]

*Table 6.* Full Pass@1 (%) by domain (target: all 13 domains, 961 tasks).

| Domain | Tasks | Pass@1 (%) |
|---|---|---|
| schematic_netlist | 72 | — |
| spice_sim | 72 | — |
| physical_verify | 61 | — |
| layout_aether | 35 | — |
| layout_virtuoso | 33 | — |
| waveform | 33 | — |
| analog_context | 30 | — |
| characterization | 25 | — |
| digital | 24 | 0.0 |
| pg_layout | 23 | — |
| pnr_signoff | 23 | 18.8 |
| eda_other | 17 | 28.6 |
| verification | 14 | 80.0 |
| **Total** | **462+** | **28.1 (partial)** |

## B.2. Per-Model Results

[TODO: multi-model comparison. Currently only DeepSeek-V4-Pro has data; GLM-5.2 / Kimi-K2.6 / DeepSeek-V4-Flash are pending.]

## B.3. Failure Taxonomy by Domain

[TODO: failure-category breakdown by domain (no-output / partial-output / format), stacked bar chart + table. Current data: overall 60/25/15; digital 100/0/0.]

*Table 7.* Pass@1 (%) by model backbone.

| Backbone | Pass@1 (%) |
|---|---|
| DeepSeek-V4-Pro | 28.1 |
| GLM-5.2 | — |
| Kimi-K2.6 | — |
| DeepSeek-V4-Flash | — |
| Doubao-Seed-2.0-Pro | — |

## B.4. Token Usage and Cost Analysis

[TODO: per-task average input/output tokens, average number of tool calls, and average wall-clock time, computed from status.jsonl.]

# C. Experimental Details

## C.1. Cline Agent Configuration

**Version.** Cline v3.0.51 (Cline Team, 2026), open-source.

**Thinking budget.** thinking=low (default). We chose low over medium/high to control token cost across 961 tasks; the trade-off is reduced reasoning depth. A thinking-budget ablation is planned for the camera-ready version.

**Auto-approve.** All tool calls auto-approved (no human-in-the-loop). The agent may iterate freely within the 2400s timeout.

**MCP bridge.** The eda_flow MCP server (port 18891) exposes 8 tools: open_session, close_session, declare_done, run_commands, run_code, read_file, write_file, list_files. The agent must call open_session first to obtain a workdir, then pass it to all subsequent calls.

## C.2. API Gateway Configuration

All models accessed via an OpenAI-compatible API gateway (agi-gateway). The providers.json configuration (with API keys masked):

```
{
  "baseURL": "http://agi-gateway.cxmt.com/cloud/v1"
  "apiKey": "<MASKED>",
  "model": "deepseek-v4-pro-cloud"
}
```

We verify API reachability with a /chat/completions health check before each batch run; a 403 response indicates key expiry.

## C.3. Anti-Leakage Hook Implementation

The PreToolUse hook is a shell script invoked before every tool call. It enforces:

- **Path whitelist**: only paths under the evaluation workdir root are accessible.
- **Answer-key blocklist**: any path matching `check.py`, `_check.py`, `reference/`, `hidden/`, `_reference_check.py` is blocked.
- **Tool whitelist**: only `read_file`, `write_to_file`, `eda_flow/*` MCP tools, and basic shell commands are allowed. Code-search tools (`grep`, `find` outside the workdir) are blocked.
- **Blocked-call response**: the hook returns the string "ACCESS RESTRICTED" to the agent, which is logged but does not terminate the session.

We count blocked calls per session (`hook_blocks` field in `status.jsonl`) to verify the hook is firing.

### C.4. Judge Implementation

The judge runs *after* the agent exits, in a separate directory:

1. Unpack `submit.zip` into a fresh workdir.
2. Invoke `check.py` with `(deliverable_path, workdir)` arguments.
3. `check.py` uses only Python standard library (no EDA license required).
4. A task passes iff all $k \geq 8$ assertions hold.

The `test_rationale` field documents each assertion in human-readable form ("k assertions: (1), (2), ...") for interpretability and audit.

### C.5. Open-Source EDA Extension Roadmap

To address the reproducibility limitation of commercial EDA tools, we plan an open-source-EDA extension mapping each domain to an open-source equivalent:

- **Digital synthesis**: Synopsys DC $\rightarrow$ Yosys + OpenROAD.
- **SPICE simulation**: HSPICE/Spectre $\rightarrow$ ngspice/Xyce.
- **Physical verification**: Calibre $\rightarrow$ KLayout + Magic.
- **Layout**: Virtuoso/Aether $\rightarrow$ KLayout + OpenROAD.

The open-source extension will share the task schema, harness, and anti-leakage protocol of the commercial benchmark, enabling cross-ecosystem comparison.